

Bit Manipulation Templates

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
1 3 6	Single Number	Array where every element appears twice except one. Find it.	XOR is its own inverse, so every duplicated pair cancels to 0 and only the lone element is left standing.	$O(n)$	$O(1)$	Standard
2 6 8	Missing Number	Array of n distinct numbers from $[0, n]$ with one missing. Find it.	XOR-ing every index against every value pairs each present number with its index — the missing number's index has no partner to cancel.	$O(n)$	$O(1)$	XOR each index <code>i</code> with <code>nums[i]</code> . Indices $0..n$ XOR'd with the n elements — the missing index has no pair.
1 3 7	Single Number II	Every element appears three times except one. Find it.	XOR cancels pairs (mod 2); here we need a counter mod 3, so two state bits (<code>ones</code> , <code>twos</code>) cycle each bit through $1 \rightarrow 2 \rightarrow 0$ and the survivor sits in <code>ones</code> .	$O(n)$	$O(1)$	Two-state XOR machine. <code>ones</code> tracks bits seen $1 \bmod 3$ times, <code>twos</code> tracks bits seen $2 \bmod 3$ times. When a bit reaches 3, it clears from both.
2 6 0	Single Number III	Two elements appear exactly once; all others appear twice. Find both.	XOR of all leaves <code>a ^ b</code> ; any set bit in it is a bit where <code>a</code> and <code>b</code> differ, so it splits the array into two groups each holding one unique.	$O(n)$	$O(1)$	XOR all $\rightarrow$ get <code>a ^ b</code> . Use lowest set bit of <code>a ^ b</code> to partition nums into two groups (one containing <code>a</code> , one containing <code>b</code>). XOR each group independently.
1 9 1	Number of 1 Bits	Return the number of set bits in an unsigned integer.	Each <code>n & (n-1)</code> erases exactly one set bit, so the loop runs once per set bit — no need to scan all 32 positions.	$O(k)$ k = number of set bits	$O(1)$	Standard
3 3 8	Counting Bits	Return an array <code>result</code> where <code>result[i]</code> = number of 1 bits in <code>i</code> , for <code>i</code> in $[0, n]$.	<code>i</code> has the same set bits as <code>i >> 1</code> plus possibly its own last bit, so reuse the already-computed smaller answer instead of recounting.	$O(n)$	$O(n)$	DP relation — <code>i >> 1</code> is <code>i</code> with last bit removed (already computed), plus the last bit <code>i & 1</code> .
1 9 0	Reverse Bits	Reverse the 32 bits of an unsigned integer.	Peel the lowest bit off <code>n</code> and stack it onto <code>result</code> from the bottom — after 32 shifts the bit order is fully mirrored.	$O(32)$ = $O(1)$	$O(1)$	Fixed 32 iterations — each time take the LSB of <code>n</code> , append it to <code>result</code> , then shift both.
2 0 1	Bitwise AND of Numbers Range	Return the AND of all numbers in <code>[left, right]</code> .	AND only keeps bits that are 1 in every number; the low bits flip somewhere in the range, so only the common high-bit prefix of <code>left</code> and <code>right</code> survives.	$O(\log n)$	$O(1)$	right-shift both until equal to find shared prefix; shift back.
3 7 1	Sum of Two Integers	Return <code>a + b</code> without using <code>+</code> or <code>-</code> .	Addition splits into a carry-free sum (XOR) and the carries (AND shifted left); feed the carry back until nothing carries over.	$O(32)$ = $O(1)$	$O(1)$	XOR computes bit sum without carry; AND shifted left computes carry. Repeat until no carry.
3 1 8	Maximum Product of Word Lengths	Given a list of words, find the maximum product <code>len(a) * len(b)</code> where <code>a</code> and <code>b</code> share no common letters.	Compress each word's letter set into a 26-bit integer so "share a letter?" becomes a single AND, replacing slow character-by-character comparison.	$O(n^2 + n \cdot k)$ k = avg word length	$O(n)$	encode each word as a 26-bit integer where bit <code>i</code> = 1 if letter 'a' + <code>i</code> appears. Two words share a letter iff their bitmasks have any common bit (<code>mask[i] & mask[j] != 0</code>).
2 3 1	Power of Two	Given an integer <code>n</code> , return true if it is a power of two.	A power of two is a single 1 bit, and <code>n & (n-1)</code> clears that one bit to 0 — anything else leaves bits behind.	$O(1)$	$O(1)$	single-bit property of powers of 2
3 4 2	Power of Four	Given an integer <code>n</code> , return true if it is a power of four.	Powers of four are powers of two (one set bit) whose bit sits at an even position; masking against the odd-position mask <code>0xAAAAAAAA</code> must give 0.	$O(1)$	$O(1)$	Power of four must be power of two (one set bit) AND that bit must be at an even bit position (0, 2, 4, 6, ...). Mask <code>0xAAAAAAAA</code> has bits set at ALL odd positions; if <code>n & 0xAAAAAAAA == 0</code> , the set bit is at an even position.
2 8 7	Find the Duplicate Number	Given an array of $n+1$ integers where each is in $[1, n]$, find the duplicate. No extra space, no modifying the array.	Following <code>i $\rightarrow$ nums[i]</code> turns the array into a linked list; a duplicate value means two indices point to the same node, forming a cycle whose entry is the duplicate.	$O(n)$	$O(1)$	Floyd's Cycle Detection. Treat the array as a linked list where <code>nums[i]</code> is the next node. Since there's a duplicate, two indices point to the same value $\rightarrow$ creates a cycle. Find cycle entry = duplicate.
4 0 5	Convert a Number to Hexadecimal	Given an integer <code>num</code> , return its hexadecimal representation as a lowercase string. For negative numbers use two's complement.	Hex is base 16, so each group of 4 bits maps to one hex digit; mask the lowest 4 bits with <code>num & 15</code> and shift right 4 at a time, using an unsigned shift so the two's-complement sign bit fills with zeros.	$O(8)$ = $O(1)$	$O(1)$	Standard
4 6 1	Hamming Distance	Given two integers <code>x</code> and <code>y</code> , return the Hamming distance — the number of bit positions where they differ.	XOR sets exactly the bits where <code>x</code> and <code>y</code> differ, so the answer is just the number of set bits in <code>x ^ y</code> .	$O(k)$ k = differing bits	$O(1)$	Standard
4 7 6	Number Complement	Given a positive integer <code>num</code> , return its complement: flip every bit up to and including the most significant set bit.	Build a mask of all 1s spanning the bit width of <code>num</code> (from the MSB down to bit 0), then XOR — flipping exactly the meaningful bits while leaving the leading zeros untouched.	$O(\log \text{num})$	$O(1)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
957	Prison Cells After N Days	8 prison cells (cells[i] is 0 or 1) update each day: a cell becomes 1 if both neighbors are equal, else 0. The two end cells always become 0 (they lack two neighbors). Return the state after n days.	With only 8 cells the state space is finite, so the configuration must cycle; detect the cycle length and reduce n modulo it to avoid simulating huge day counts.	O(min(n, cycle) · 8)	O(states)	Standard

Canonical Template

```
// MENTAL MODEL: bits are a set/counter – XOR cancels duplicates, AND/OR/shift edit individual bits.
// WHEN: "appears once/odd among pairs", "count set bits", "no shared letters", "power of 2/4".
// — XOR CANCEL — pairs cancel; lone element survives
int result = 0;
for (int num : nums) result ^= num;

// — BIT OPERATIONS —
boolean isSet = ((n >> i) & 1) == 1; // check if bit i is set
int set = n | (1 << i); // set bit i
int clear = n & ~(1 << i); // clear bit i
int toggle = n ^ (1 << i); // toggle bit i
int lowest = n & (-n); // isolate lowest set bit
boolean isPow2 = n > 0 && (n & (n-1)) == 0;

// — COUNT SET BITS — Brian Kernighan: each iteration removes one set bit
// WHY n & (n-1) clears the lowest set bit:
// n = ...1000 (lowest set bit at this position)
// n-1 = ...0111 (borrow flips that bit to 0 and all lower bits to 1)
// n&(n-1)=...0000 (AND keeps higher bits, wipes the lowest set bit + below)
int count = 0;
while (n != 0) { count++; n &= (n - 1); } // n & (n-1) clears lowest set bit

// — BITMASK AS SET — 26-bit integer represents presence of 26 letters
int mask = 0;
for (char c : word.toCharArray()) mask |= (1 << (c - 'a'));
// check no overlap between two words:
if ((mask[i] & mask[j]) == 0) { /* no shared letter */ }
```

Variations

```
// VARIATION 1: mod-3 state machine (Single Number II)
// MENTAL MODEL: extend XOR from mod-2 (cancel pairs) to mod-3 (cancel triples);
// ones/twos track bits seen 1 or 2 times mod 3.
// Instead of XOR canceling pairs (mod 2), cancel triples (mod 3)
// ones = bits seen exactly 1 mod 3 times; twos = bits seen exactly 2 mod 3 times
int ones = 0, twos = 0;
for (int num : nums) {
    ones = (ones ^ num) & ~twos;    // ← add to ones if not already in twos
    twos = (twos ^ num) & ~ones;    // ← add to twos if not already in ones
}
// After all: ones holds the element appearing once (0 mod 3 remainder = survive)

// VARIATION 2: split XOR into two groups (Single Number III)
// Two unique elements a, b. XOR of all = a ^ b.
// Use lowest set bit of (a^b) to split nums into two groups → each group has one unique.
int xor = 0;
for (int num : nums) xor ^= num;    // xor = a ^ b
int diff = xor & (-xor);            // ← lowest set bit that differs between a and b
int a = 0;
for (int num : nums)
    if ((num & diff) != 0) a ^= num; // ← XOR one group → one unique element
int b = xor ^ a;                    // ← the other unique element

// VARIATION 3: DP relation (Counting Bits)
// dp[i] = dp[i/2] + last bit of i
// i >> 1 = i without last bit (same or fewer ones); last bit = i & 1
int[] dp = new int[n + 1];
for (int i = 1; i <= n; i++)
    dp[i] = dp[i >> 1] + (i & 1);    // ← right-shift halves the problem

// VARIATION 4: common prefix (Bitwise AND of Range)
// AND of any range [left, right]: bits that differ between left and right get cleared.
// Right-shift both until equal → find shared high-bit prefix → shift back.
int shift = 0;
while (left != right) { left >>= 1; right >>= 1; shift++; }
return left << shift;                // ← shift back to original magnitude

// VARIATION 5: carry loop (Sum of Two Integers)
// XOR gives bit sum without carry; AND shifted left gives carry.
// Repeat until no carry remains.
int add(int a, int b) {
    while (b != 0) {
        int carry = a & b;            // ← carry bits
        a = a ^ b;                    // ← partial sum (no carry)
        b = carry << 1;                // ← carry propagated one position left
    }
    return a;
}
```

Part 1 — XOR Cancel

#136 Single Number

Description: Array where every element appears twice except one. Find it.

Intuition: XOR is its own inverse, so every duplicated pair cancels to 0 and only the lone element is left standing.

Algorithm: XOR all — every pair cancels ($a \oplus a = 0$), leaving the single element.

```
class Solution {
    public int singleNumber(int[] nums) {
        int result = 0;
        for (int num : nums) result ^= num;    // pairs cancel; lone element survives
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#268 Missing Number

Description: Array of n distinct numbers from $[0, n]$ with one missing. Find it.

Intuition: XOR-ing every index against every value pairs each present number with its index — the missing number's index has no partner to cancel.

Variation: XOR each index i with $\text{nums}[i]$. Indices $0..n$ XOR'd with the n elements — the missing index has no pair.

```
class Solution {
    public int missingNumber(int[] nums) {
        int result = nums.length;           // start with n (last index)
        for (int i = 0; i < nums.length; i++)
            result ^= i ^ nums[i];          // ← VARIATION: XOR index with value
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#137 Single Number II

Description: Every element appears three times except one. Find it.

Intuition: XOR cancels pairs (mod 2); here we need a counter mod 3, so two state bits (`ones` , `twos`) cycle each bit through $1 \rightarrow 2 \rightarrow 0$ and the survivor sits in `ones`.

Variation: Two-state XOR machine. `ones` tracks bits seen 1 mod 3 times, `twos` tracks bits seen 2 mod 3 times. When a bit reaches 3, it clears from both.

```
class Solution {
    public int singleNumber(int[] nums) {
        int ones = 0, twos = 0;
        for (int num : nums) {
            ones = (ones ^ num) & ~twos;    // ← VARIATION: add to ones only if not in twos
            twos = (twos ^ num) & ~ones;    // ← VARIATION: add to twos only if not in ones
        }
        return ones;                       // ones holds the element seen exactly once
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#260 Single Number III

Description: Two elements appear exactly once; all others appear twice. Find both.

Intuition: XOR of all leaves $a \oplus b$; any set bit in it is a bit where a and b differ, so it splits the array into two groups each holding one unique.

Variation: XOR all $\rightarrow$ get $a \oplus b$. Use lowest set bit of $a \oplus b$ to partition nums into two groups (one containing a , one containing b). XOR each group independently.

```
class Solution {
    public int[] singleNumber(int[] nums) {
        int xor = 0;
        for (int num : nums) xor ^= num;    // xor = a ^ b

        int diff = xor & (-xor);            // ← VARIATION: isolate lowest bit that differs
        int a = 0;
        for (int num : nums)
            if ((num & diff) != 0) a ^= num; // ← VARIATION: XOR only one group
        return new int[]{a, xor ^ a};        // b = (a^b) ^ a
    }
}
```

Time $O(n)$ | **Space** $O(1)$

Part 2 — Bit Counting

#191 Number of 1 Bits

Description: Return the number of set bits in an unsigned integer.

Intuition: Each $n \& (n-1)$ erases exactly one set bit, so the loop runs once per set bit — no need to scan all 32 positions.

Algorithm: Brian Kernighan — `n & (n-1)` clears the lowest set bit each iteration.

```
public class Solution {
    public int hammingWeight(int n) {
        int count = 0;
        while (n != 0) {
            count++;
            n &= (n - 1);    // ← clears lowest set bit; loop runs exactly hamming-weight times
        }
        return count;
    }
}
```

Time $O(k)$ k = number of set bits | **Space** $O(1)$

#338 Counting Bits

Description: Return an array `result` where `result[i]` = number of 1 bits in `i`, for `i` in $[0, n]$.

Intuition: `i` has the same set bits as `i >> 1` plus possibly its own last bit, so reuse the already-computed smaller answer instead of recounting.

Variation: DP relation — `i >> 1` is `i` with last bit removed (already computed), plus the last bit `i & 1`.

```
class Solution {
    public int[] countBits(int n) {
        int[] dp = new int[n + 1];
        for (int i = 1; i <= n; i++)
            dp[i] = dp[i >> 1] + (i & 1);    // ← VARIATION: dp[i/2] + last bit
        return dp;
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#190 Reverse Bits

Description: Reverse the 32 bits of an unsigned integer.

Intuition: Peel the lowest bit off `n` and stack it onto `result` from the bottom — after 32 shifts the bit order is fully mirrored.

Variation: Fixed 32 iterations — each time take the LSB of `n`, append it to `result`, then shift both.

```
public class Solution {
    public int reverseBits(int n) {
        int result = 0;
        for (int i = 0; i < 32; i++) {    // ← VARIATION: always 32 iterations (fixed-width)
            result = (result << 1) | (n & 1); // append LSB of n to result
            n >>= 1;
        }
        return result;
    }
}
```

Time $O(32) = O(1)$ | **Space** $O(1)$

Part 3 — Range / Arithmetic

#201 Bitwise AND of Numbers Range

Description: Return the AND of all numbers in `[left, right]`.

Intuition: AND only keeps bits that are 1 in every number; the low bits flip somewhere in the range, so only the common high-bit prefix of `left` and `right` survives.

Key insight: any two adjacent numbers in the range differ in at least the lowest bit. AND of a range clears all bits that differ across any pair in the range. The result is the common high-bit prefix of `left` and `right`.

Variation: right-shift both until equal to find shared prefix; shift back.

```

class Solution {
    public int rangeBitwiseAnd(int left, int right) {
        int shift = 0;
        while (left != right) {                // ← VARIATION: shift until common prefix found
            left  >>= 1;
            right >>= 1;
            shift++;
        }
        return left << shift;                  // ← restore to original magnitude
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#371 Sum of Two Integers

Description: Return $a + b$ without using $+$ or $-$.

Intuition: Addition splits into a carry-free sum (XOR) and the carries (AND shifted left); feed the carry back until nothing carries over.

Variation: XOR computes bit sum without carry; AND shifted left computes carry. Repeat until no carry.

```

class Solution {
    public int getSum(int a, int b) {
        while (b != 0) {
            int carry = a & b;                // ← VARIATION: carry = bits where both are 1
            a = a ^ b;                        // ← VARIATION: partial sum (no carry propagation)
            b = carry << 1;                   // ← VARIATION: carry propagates one bit left
        }
        return a;
    }
}

```

Time $O(32) = O(1)$ | **Space** $O(1)$

Part 4 — Bitmask as Set

#318 Maximum Product of Word Lengths

Description: Given a list of words, find the maximum product $\text{len}(a) * \text{len}(b)$ where a and b share no common letters.

Intuition: Compress each word's letter set into a 26-bit integer so "share a letter?" becomes a single AND, replacing slow character-by-character comparison.

Variation: encode each word as a 26-bit integer where bit $i = 1$ if letter $'a' + i$ appears. Two words share a letter iff their bitmasks have any common bit ($\text{mask}[i] \& \text{mask}[j] \neq 0$).

```

class Solution {
    public int maxProduct(String[] words) {
        int n = words.length;
        int[] masks = new int[n];
        for (int i = 0; i < n; i++)
            for (char c : words[i].toCharArray())
                masks[i] |= (1 << (c - 'a'));    // ← VARIATION: 26-bit bitmask per word

        int max = 0;
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                if ((masks[i] & masks[j]) == 0) // ← VARIATION: no shared letters = AND is zero
                    max = Math.max(max, words[i].length() * words[j].length());

        return max;
    }
}

```

Time $O(n^2 + n \cdot k)$ $k = \text{avg word length}$ | **Space** $O(n)$

Bit Trick Cheat Sheet

```
n & (n-1)      // clear lowest set bit → count bits, check power of 2
n & (-n)       // isolate lowest set bit → split into groups
n ^ n = 0      // same value cancels → XOR pairs
n ^ 0 = n      // identity → XOR with index
(n >> i) & 1    // check bit i
n | (1 << i)    // set bit i
n & ~(1 << i)   // clear bit i
n ^ (1 << i)    // toggle bit i
i >> 1         // divide by 2 (fast)
i & 1         // last bit (0 = even, 1 = odd)

// --- Divisibility by powers of two ---
(n & 1) == 0    // divisible by 2 (even) ← LSB is the 2^0 place
(n & 1) == 1    // NOT divisible by 2 (odd)
(n & ((1 << k) - 1)) // remainder of n / 2^k ; == 0 means divisible by 2^k
(n & 3) == 0    // divisible by 4 (k = 2)
(n & 7) == 0    // divisible by 8 (k = 3)
// Note: n & 1 is safe for NEGATIVE numbers (two's complement) where
// n % 2 returns -1 for odd negatives. Prefer & 1 for parity.
```

Pattern Chooser

Signal	Technique	Time
"appears odd/once, rest appear even/twice"	XOR all	O(n)
"appears once, rest appear 3 times"	mod-3 state machine (ones , twos)	O(n)
"two unique elements"	XOR → split by xor & (-xor)	O(n)
Count set bits	Brian Kernighan n &= (n-1)	O(k) k = set bits
Count bits for all 0..n	DP: dp[i] = dp[i>>1] + (i&1)	O(n)
AND of a range	Right-shift to find common prefix	O(log n)
Add without +	XOR + carry loop	O(1) (32 iters)
"no shared characters between two strings"	26-bit bitmask, check AND == 0	O(n²) pairwise
"is n a power of two?"	n > 0 && (n & (n-1)) == 0	O(1)
"is n a power of four?"	Power of 2 AND set bit at even position: (n & 0xAAAAAAAA) == 0	O(1)
"find duplicate, no extra space, no modification"	Floyd's cycle detection on array-as-linked-list	O(n)

Part 5 — Power Checks & Cycle Detection

#231 Power of Two

Description: Given an integer `n` , return true if it is a power of two.

Intuition: A power of two is a single 1 bit, and `n & (n-1)` clears that one bit to 0 — anything else leaves bits behind.

Algorithm: A power of two has exactly one set bit. Check `n > 0` and `n & (n-1) == 0` (clearing the lowest set bit results in 0 only if there was one bit).

```
class Solution {
    public boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0; // ← VARIATION: single-bit property of powers of 2
    }
}
```

Time O(1) | Space O(1)

#342 Power of Four

Description: Given an integer `n` , return true if it is a power of four.

Intuition: Powers of four are powers of two (one set bit) whose bit sits at an even position; masking against the odd-position mask `0xAAAAAAAA` must give 0.

Variation: Power of four must be power of two (one set bit) AND that bit must be at an even bit position (0, 2, 4, 6, ...). Mask `0xAAAAAAAA` has bits set at ALL odd positions; if `n & 0xAAAAAAAA == 0`, the set bit is at an even position.

```
class Solution {
    public boolean isPowerOfFour(int n) {
        // 0xAAAAAAAA = 1010 1010 ... 1010 (bits at odd positions 1,3,5,...31)
        return n > 0 && (n & (n-1)) == 0 // ← power of 2: exactly one set bit
            && (n & 0xAAAAAAAA) == 0; // ← VARIATION: set bit at even position (4^k has bit at position 2k)
    }
}
```

Time $O(1)$ | **Space** $O(1)$

#287 Find the Duplicate Number

Description: Given an array of `n+1` integers where each is in `[1, n]`, find the duplicate. No extra space, no modifying the array.

Intuition: Following `i → nums[i]` turns the array into a linked list; a duplicate value means two indices point to the same node, forming a cycle whose entry is the duplicate.

Variation: Floyd's Cycle Detection. Treat the array as a linked list where `nums[i]` is the next node. Since there's a duplicate, two indices point to the same value → creates a cycle. Find cycle entry = duplicate.

```
class Solution {
    public int findDuplicate(int[] nums) {
        // Phase 1: find intersection point in cycle
        int slow = nums[0], fast = nums[0];
        do {
            slow = nums[slow]; // ← VARIATION: Floyd's cycle, not XOR
            fast = nums[nums[fast]];
        } while (slow != fast);
        // Phase 2: find cycle entry (= duplicate)
        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

Additional Reference Problems

#405 Convert a Number to Hexadecimal

Description: Given an integer `num`, return its hexadecimal representation as a lowercase string. For negative numbers use two's complement.

Intuition: Hex is base 16, so each group of 4 bits maps to one hex digit; mask the lowest 4 bits with `num & 15` and shift right 4 at a time, using an unsigned shift so the two's-complement sign bit fills with zeros.

Algorithm: Repeatedly take `num & 0xF` to read the lowest nibble, map it to a hex char, then unsigned-right-shift `num >>= 4`. Stop when `num` becomes 0.

```
class Solution {
    public String toHex(int num) {
        if (num == 0) {
            return "0";
        }
        char[] digits = "0123456789abcdef".toCharArray();
        StringBuilder builder = new StringBuilder();
        while (num != 0) {
            int nibble = num & 15; // ← lowest 4 bits = one hex digit
            builder.append(digits[nibble]);
            num >>= 4; // unsigned shift handles negatives (two's complement)
        }
        return builder.reverse().toString();
    }
}
```

Time $O(8) = O(1)$ | **Space** $O(1)$

#461 Hamming Distance

Description: Given two integers `x` and `y`, return the Hamming distance — the number of bit positions where they differ.

Intuition: XOR sets exactly the bits where `x` and `y` differ, so the answer is just the number of set bits in `x ^ y`.

Algorithm: Compute `x ^ y`, then count its set bits with Brian Kernighan (`n & (n-1)` clears the lowest set bit each iteration).

```
class Solution {
    public int hammingDistance(int x, int y) {
        int n = x ^ y;           // differing bits become 1
        int count = 0;
        while (n != 0) {
            count++;
            n &= (n - 1);         // ← clears lowest set bit each iteration
        }
        return count;
    }
}
```

Time $O(k)$ k = differing bits | **Space** $O(1)$

#476 Number Complement

Description: Given a positive integer `num`, return its complement: flip every bit up to and including the most significant set bit.

Intuition: Build a mask of all 1s spanning the bit width of `num` (from the MSB down to bit 0), then XOR — flipping exactly the meaningful bits while leaving the leading zeros untouched.

Algorithm: Grow a mask `1 << i` until it covers all bits of `num` (`mask < num`), forming `(1 << bitLength) - 1`; XOR `num` with that mask to flip only the relevant bits.

```
class Solution {
    public int findComplement(int num) {
        int mask = 0;
        int i = 0;
        while ((1 << i) <= num) {           // ← extend mask to num's bit width
            mask |= (1 << i);               // set bit i in the all-ones mask
            i++;
        }
        return num ^ mask;                  // XOR flips every bit within the mask
    }
}
```

Time $O(\log \text{num})$ | **Space** $O(1)$

#957 Prison Cells After N Days

Description: 8 prison cells (`cells[i]` is 0 or 1) update each day: a cell becomes 1 if both neighbors are equal, else 0. The two end cells always become 0 (they lack two neighbors). Return the state after `n` days.

Intuition: With only 8 cells the state space is finite, so the configuration must cycle; detect the cycle length and reduce `n` modulo it to avoid simulating huge day counts.

Algorithm: Encode the 8 cells as an 8-bit integer. Simulate one day with bit operations (a cell is 1 iff its neighbors match, i.e. their XOR is 0), recording seen states in a map. On the first repeat, reduce remaining days modulo the cycle length, then finish the leftover days.

```

import java.util.HashMap;
import java.util.Map;

class Solution {
    public int[] prisonAfterNDays(int[] cells, int n) {
        int state = 0;
        for (int i = 0; i < 8; i++) {
            if (cells[i] == 1) {
                state |= (1 << i);          // pack cells into an 8-bit integer
            }
        }
        Map<Integer, Integer> seen = new HashMap<>();
        while (n > 0) {
            if (seen.containsKey(state)) {
                n %= seen.get(state) - n;    // ← cycle found: skip whole cycles
            } else {
                seen.put(state, n);
            }
            if (n > 0) {
                state = nextDay(state);
                n--;
            }
        }
        int[] result = new int[8];
        for (int i = 0; i < 8; i++) {
            result[i] = (state >> i) & 1;    // unpack bits back into cells
        }
        return result;
    }

    private int nextDay(int state) {
        int next = 0;
        for (int i = 1; i < 7; i++) {      // end cells (0 and 7) always become 0
            int left = (state >> (i - 1)) & 1;
            int right = (state >> (i + 1)) & 1;
            if ((left ^ right) == 0) {      // neighbors equal → cell becomes 1
                next |= (1 << i);
            }
        }
        return next;
    }
}

```

Time $O(\min(n, \text{cycle}) \cdot 8)$ | **Space** $O(\text{states})$